

Padé vs Richardson for the K=3 lookup function: rational extrapolation does NOT beat polynomial

Fixed-Point Factory — two-kernel extrapolation test

June 2026

Abstract

Tested: replace Richardson (polynomial in h^2) with Padé rational approximants $\text{Padé}(0,1) = \frac{a}{1+bh^2}$ and $\text{Padé}(1,1) = \frac{a+bh^2}{1+ch^2}$ to extrapolate the kernel-band lookup to $h=0$. Hypothesis: rational forms handle non-analytic h -dependence better than polynomial.

Result: hypothesis wrong. On 16 saved R4 FPs at $G=11$:

- At $\tau=0.2$ (smooth regime): $\text{Padé}(0,1)$ ties Richardson R2 ($\max|\Delta| = 0.21$); $\text{Padé}(1,1)$ DEGENERATES because $\mu \approx 0.5$ over flat regions makes the rational denominator's linear system rank-deficient – returns constant 0.5.
- At $\tau=1$ (kinky regime): $\text{Padé}(0,1)$ max error ~ 0.80 to 0.91 (vs Richardson R2 max ~ 0.4); $\text{Padé}(1,1)$ max ~ 0.67 . Both WORSE than Richardson.

Mechanism. The kernel-band $\mu(h)$ does *not* have poles in h^2 . It has *jumps* – non-analytic discontinuities at h values where the band edges sweep across kinks/critical points. Padé approximants try to fit poles where there are none; the denominator $1 + bh^2$ approaches zero spuriously, giving wild extrapolations. Richardson's polynomial form is safer (can't blow up).

Conclusion. The two-kernel extrapolation idea is mechanically sound but doesn't help because the underlying $\mu(h)$ isn't pole-like. The right path to handle the non-analyticity is still **cusp-aware quadrature** (handle each topology change as an analytic singular contribution), not a rational approximant in h .

1 Cell-by-cell results

cell	Rich R2 max	Rich R4 max	Padé(0,1) max	Padé(1,1) max
$\gamma=10, \tau=0.2$	0.21	0.21	0.21	0.50
$\gamma=10, \tau=1$	0.43	0.57	0.91	0.68
$\gamma=100, \tau=0.2$	0.21	0.21	0.21	0.50
$\gamma=100, \tau=1$	0.38	0.44	0.85	0.68
$\gamma=1000, \tau=0.2$	0.21	0.21	0.21	0.50
$\gamma=1000, \tau=1$	0.38	0.44	0.81	0.68

2 Why Padé fails

For Padé(0,1) $f(h) = a/(1 + bh^2)$ fit to two points:

$$b = (A_2 - A_1)/(A_1 h_1^2 - A_2 h_2^2)$$
$$a = A_1(1 + bh_1^2)$$

If A_1, A_2 are close (smooth μ), the numerator $A_2 - A_1$ is small AND the denominator $A_1 h_1^2 - A_2 h_2^2 \approx A_{\text{common}}(h_1^2 - h_2^2)$ is the difference of two close numbers — catastrophic cancellation. The estimator is noisy.

If μ has true poles (which it doesn't here, only jumps), $b > 0$ captures them and extrapolation is exact. But for jump discontinuities, the data look like a low-degree polynomial PLUS noise from the jump; Padé treats the noise as a pole and extrapolates into a spurious singularity.

For Padé(1,1) with 3 points: 3×3 linear system in (a, b, c) becomes near-singular when the three A_i are nearly equal (flat μ); `np.linalg.solve` returns garbage that we fall back to mean of inputs, hence the 0.5 result at smooth cells.

3 Conclusion

Padé in h^2 is the wrong tool for this operator. The right path remains:

1. Detect critical points of P analytically (zeros of ∇P , solvable from Cheby coefs via 2D resultant).
2. Add an analytic kink residual to the GL quadrature near each critical p^* .

This makes the operator continuous in P and restores exponential spectral convergence at any τ .

4 Reproducibility

`projects/REZN/solver_code/highprec_dd_qd/dd_k3_pade.py`